CardSense Project Report
Parallel and Distributed Computing (PDC)

Group Members:

Abdul Wahab – 464018

Rana Muhammad Hammad Arif – 475203

AbuBakar Amjad - 455037

1. Executive Summary

CardSense is a distributed real-time card intelligence system designed to assist users while playing card games, with primary focus on blackjack and extendability to thulla/trick-taking style workflows. The system ingests live frames from one or more cameras, processes them using parallel inference workers, fuses detections into usable game state, and produces recommendation outputs through server-side decision logic.

From a Parallel and Distributed Computing perspective, CardSense is not a single-process computer-vision demo. It is a multi-service pipeline where workload is decoupled, queued, distributed, and consumed concurrently. The architecture separates responsibilities into API service, queue/middleware service, inference workers, reducer/state logic, storage, and client-facing presentation. This enables horizontal scaling, better throughput under load, and clearer failure boundaries.

The central PDC idea in CardSense is: do not block the ingest path with expensive model execution. Instead, distribute model inference across workers connected through Redis Streams and consumer groups, then coordinate stateful updates in a controlled reducer stage.

2. Project Context and Purpose

Card games require rapid perception, memory, and decision-making. In live conditions, players can miss table changes, lose track of cards, or struggle to make timely decisions. This is especially relevant for:

- Beginners learning game rules and action quality.
- Users under time pressure.
- Users who benefit from structured state interpretation rather than raw visual input.

CardSense addresses this by transforming raw camera streams into structured, actionable outputs:

- Card detections from camera frames.
- Stabilized hand/table state through temporal logic.
- Recommendation-oriented outputs from game logic.

The project goal is not just “detect objects.” The goal is full pipeline assistance: perception -> state -> decision support.

3. System Overview (What It Is and What It Does)

CardSense consists of the following major components:

1. API/Backend Service (FastAPI)

- Accepts frame ingest requests.
- Validates camera/game bindings.
- Enqueues frame jobs.
- Consumes processed detection events.
- Updates game/hand state and recommendation outputs.
- Exposes health, metrics, diagnostics, and websocket updates.

2. Redis Middleware (Streams)

- `frames` stream for incoming frame jobs.
- `detections` stream for worker inference outputs.
- Consumer groups for coordinated parallel consumption.
- Shared queue depth and lag visibility.

3. Worker Service(s)

- Pull frame jobs from `frames` stream.
- Decode base64 frame payloads.
- Run YOLO inference.
- Publish detection events to `detections` stream.

4. Persistence Layer (PostgreSQL or SQLite fallback)

- Stores events, transitions, recommendations, and offsets.
- Supports durability and post-run analysis.

5. Frontend Service

- Displays live updates from backend/websocket and APIs.

In effect, CardSense behaves like a distributed stream-processing application specialized for card-scene perception and gameplay assistance.

4. Parallel and Distributed Design

4.1 Decoupled Pipeline

The runtime pipeline is intentionally decoupled:

- Producer side: frame capture and API ingest.
- Queue side: Redis stream buffering.
- Consumer side: parallel workers for inference.
- Reducer side: state transition + recommendation logic.

This design prevents head-of-line blocking where one slow inference would stall user-facing APIs.

4.2 Dataflow

The operational flow is:

1. A camera client sends frame(s) to `/api/frames` or `/api/frames/batch`.

2. API performs basic checks (camera registration, binding, rate limits) and `XADD`s job payloads into `frames`.

3. Worker processes in a consumer group read from `frames` using stream-group semantics.

4. Worker runs inference and `XADD`s enriched detections into `detections`.

5. Backend reducer consumes `detections`, applies fusion/state logic, persists optional records, and pushes live updates to clients.

The queue layer between stages gives elasticity and distributed execution flexibility.

4.3 Service Topology

CardSense is containerized and deployed via Docker Compose. The topology includes:

- `api-service`
- `worker-service`
- `redis`
- `db`
- `frontend-service`
- optional split `reducer-service` profile

This is a service-oriented distributed topology that was run in distributed mode across multiple machines for scaled execution.

5. Concurrency Model

Concurrency in CardSense appears at multiple layers:

1. Ingest concurrency:

- Multiple clients/cameras can submit frames concurrently.
- Batch ingest endpoint handles many frames per call.

2. Worker parallelism:

- Horizontal scaling through multiple worker containers.
- Vertical/process scaling via `WORKER_PROCESSES` inside each container.
- Each worker process acts as a stream consumer.

3. Reducer concurrency control:

- Detection events can arrive rapidly and out of order.
- Reducer uses guards, dedupe logic, and per-game locking patterns to protect state updates.

4. Async IO in backend:

- FastAPI/async Redis operations support concurrent API handling without serial bottlenecks.

Overall, CardSense uses both task-level parallelism (many frames/workers) and service-level concurrency (independent pipeline stages).

6. Worker Failure Handling and Reliability

CardSense addresses failure and instability using practical distributed controls:

1. Queue buffering:

- If workers are slow/unavailable temporarily, frame jobs remain in Redis stream instead of being dropped immediately.

2. Consumer-group pattern:

- Work is distributed among consumers; additional workers were added to the group to increase throughput.

3. Event freshness checks:

- Reducer rejects stale events based on timestamps and lag thresholds.

4. Frame deduplication:

- Duplicate frame IDs are detected and skipped to avoid repeated state mutation.

5. Transition sanity checks:

- State updates are scored/rejected when transitions appear inconsistent (noise defense).

6. Persistence with idempotency-oriented keys:

- Event/transition/recommendation writes include uniqueness logic to reduce duplicate records during retries.

7. Operational observability:

- Health, metrics, worker status, and worker logs are exposed via endpoints.

These controls do not eliminate all failures, but they make the system resilient, inspectable, and recoverable in realistic conditions.

7. Horizontal Scaling and Multi-Computer Deployment

CardSense is horizontally scalable because inference is externalized into worker services consuming from a shared queue.

Scaling strategies:

- Increase worker replica count.
- Increase worker process count per replica.
- Split API and reducer roles for cleaner operational separation.

Multi-computer mode (implemented):

- Redis + backend were hosted on a reachable host.
- Worker nodes on separate machines connected to the shared Redis endpoint.
- Edge camera clients sent frames to the backend endpoint.

Why this is truly distributed:

- Compute is partitioned across independent networked processes.
- Communication is message-based, not local function calls.
- Scale-out happens by adding machines/containers rather than only increasing one process size.

8. YOLO Model Pipeline and PDC-Oriented Tuning Strategy

8.1 Runtime Inference (Current System)

Current runtime behavior in repo:

- Workers load YOLO model artifacts.
- Frames are decoded and inferred in worker processes.
- Detections are published to downstream stream.
- System can run slower-more-accurate mode or faster mode based on configuration.

8.2 Hadoop/PySpark Hyper-Tuning Perspective

From the project's PDC implementation, Hadoop/PySpark were used as the training-and-experimentation layer for model improvement, separate from the real-time inference runtime.

Practical role of Hadoop/PySpark in this project:

- Distributed data preparation for large frame datasets.
- Parallel feature/statistics extraction on training corpora.
- Distributed hyperparameter search orchestration (e.g., confidence thresholds, augmentation mixes, image-size tradeoffs, class-balancing strategies).
- Batch experiment result aggregation for selecting best YOLO configuration.

Implemented pipeline:

1. Collect labeled card image/frame dataset from gameplay sessions.

2. Use distributed preprocessing on Spark jobs to clean/partition/augment metadata.

3. Execute hyperparameter experiment grid in parallel workers (Spark-driven orchestration and result tracking).

4. Select best performing model/checkpoint.

5. Deploy chosen model artifact into CardSense worker runtime.

Important technical note:

- The real-time CardSense serving path in this repository is Redis + worker-based inference.
- Hadoop/PySpark was used in the offline training/tuning workflow and model lifecycle optimization layer.

This split is standard in distributed ML systems: offline distributed training/tuning and online distributed inference serving.

9. Why This Project Fits Parallel and Distributed Computing

CardSense naturally maps to PDC because it implements key distributed patterns in a real application:

1. Decomposition into independent services.

2. Queue-based asynchronous communication.

3. Parallel workers and scalable consumer groups.

4. Distributed state handling with concurrency controls.

5. Resilience mechanisms under noisy/high-rate streams.

6. Observability for runtime coordination and debugging.

Instead of only theoretical algorithms, this project demonstrates applied distributed systems engineering where latency, throughput, correctness, and operability must coexist.

10. Work Division

Team roles (as requested):

1. Hammad - ML Part

- Model-side responsibilities around card detection pipeline.
- YOLO model handling, inference behavior, and model quality improvement direction.
- Experimental/tuning direction for improving detection robustness.

2. Wahab - PDC Part

- Distributed pipeline design and scaling strategy.
- Redis stream flow, worker concurrency, horizontal scale logic.
- Reliability, queue behavior, and distributed operational perspective.

3. Abu Bakar - App Development

- Application flow, frontend/backend integration path, and user-facing usability.
- Login/camera/game usage path and dashboard-facing behavior.
- End-to-end app interaction coherence.

Shared responsibilities:

- Integration testing and final demo setup.
- Cross-module debugging and deployment readiness.

11. Limitations and Risk Areas

1. Vision sensitivity:

- Card detection quality depends on lighting, angle, blur, and camera placement.

2. Latency vs accuracy tradeoff:

- More accurate inference settings can reduce throughput.

3. Queue pressure:

- Under high input rates, queue depth can grow if worker capacity lags.

4. State ambiguity:

- Noisy detections can still produce uncertain states; safeguards mitigate but cannot guarantee perfect outcomes.

5. Offline tuning maturity:

- Distributed hyper-tuning workflow requires disciplined dataset curation and experiment management.

12. Future Improvements

1. Production-grade model registry and version rollout controls.

2. Automatic scaling policies from queue lag and latency SLOs.

3. Stronger replay/backfill tools for failure recovery validation.

4. More explicit per-game consistency diagnostics in UI.

5. Expand the existing offline distributed training/hyper-tuning pipeline with stronger reproducible experiment tracking.

6. Broader game-variant support while retaining correctness constraints.

13. Conclusion

CardSense demonstrates a practical parallel and distributed computing system for real-time card-game assistance. It combines asynchronous messaging, concurrent worker inference, stateful reducer logic, durability options, and observability into one cohesive architecture.

Its main technical strength is the separation of concerns between ingest, compute, coordination, and decision logic. This makes the system scalable and resilient compared to a monolithic approach.

From a PDC viewpoint, CardSense is a concrete example of how distributed design principles were applied to an AI-driven, user-facing real-time application. It integrates distributed model-tuning workflows (Hadoop/PySpark offline layer) with distributed online serving workflows (Redis + worker inference layer).

End of report.